The realistic primitive equation model pe_realistic

pe_realistic solves the same equations as pe_simple with the same time stepping. This text describes only the differences to pe_simple. Everything that is not mentioned here is as described in the documentation of pe_simple.

The differences are the following. The grid spacings may vary in all three directions. The horizontal coordinates may be longitude and latitude on the sphere, as in pyOM2. The equation of state may be the one of TEOS-10. Isoneutral mixing is available. Horizontal momentum friction uses the discrete operators of pyOM2. Tracer advection can be taken out of the Adams–Bashforth step.

pyOM2 is the reference for all discrete operators that are new here. They were ported from pyOM2 and compared with pyOM2 in twin experiments. The tests and their results are listed in `pyOM3/pe_realistic/README_test_pe_realistic.md`.

The domain decomposition, the halo exchange and the iterative solver for the surface pressure are not part of the package. They are imported from pe_simple. The grid is built by the class `grid` in `grid.py`.

Grid

The model domain consists of $n_x \times n_y \times n_z$ cells. The horizontal coordinates x and y are either Cartesian coordinates in metres or longitude and latitude in degrees. The attribute `coord_degree` selects longitude and latitude. The vertical coordinate z is in metres.

The widths of the cells are given by the attributes `dxt`, `dyt` and `dzt`. Each of them is either a number or an array of length n_x, n_y or n_z without halo points. The numbers `dx`, `dy` and `dz` of pe_simple are accepted instead. Without either of them the model stops with an error.

A cell has its centre at a T point and its faces at U points. In x the cell i has the centre x^t_i, the width Δx^t_i and the eastern face x^u_i. The distance between the centres x^t_i and x^t_{i+1} is Δx^u_i. The same holds in y with y^t_j, Δy^t_j, y^u_j, Δy^u_j. In z the face on top of cell k is the W point z^w_k, and the distance between the centres z^t_k and z^t_{k+1} is Δz^w_k.

The grid is built from the widths as in pyOM2 (`u_centered_grid` of `numerics.f90`). The faces are summed from the widths. The centres follow from the faces by

$$x^u_i = x^u_{i-1} + \Delta x^t_i \ , \quad x^t_{i+1} = 2x^u_i - x^t_i \ , \quad \Delta x^u_i = x^t_{i+1} - x^t_i \tag{1}$$

with the start $x^t_0 = x^u_0 - \Delta x^t_0/2$ and the end $\Delta x^u_n = 2\Delta x^t_n - \Delta x^u_{n-1}$ at the last cell n. This recursion keeps the discrete gradient and divergence adjoint on a stretched grid.

On a stretched grid x^t_i is not the midpoint between the faces x^u_{i-1} and x^u_i. This follows from Eq. (1). It matters wherever a depth is taken from z^t, for instance in the equation of state below.

In x and y the widths are first extended by the halo points. In a periodic direction the halo widths are copies of their periodic images. In a closed direction they repeat the width of the edge cell. The recursion Eq. (1) then runs over the extended array. The first

interior U point is placed at `x_origin` and `y_origin` (default 0). In a periodic direction the halo coordinates are copies of the coordinates of their periodic images. They are used only for the metric factors below.

In z the recursion Eq. (1) runs over the n_z interior cells only, as in pyOM2. The coordinates are then extended linearly into the halo, and the halo widths repeat the edge widths. The array `dzt` is given from the bottom to the surface. The cell $k =$ `halo` is the lowest cell and the cell $k =$ `Nz-halo-1` is the surface cell, as in pe_simple.

The end of the recursion, $\Delta z^w_n = 2\Delta z^t_n - \Delta z^w_{n-1}$, can be zero or negative. This happens for instance when Δz^t grows by a constant ratio from cell to cell. The model then stops with an error. Δz^t must therefore vary smoothly.

The origin of z is the bottom face of the lowest cell by default. z is then the height above the bottom, as in pe_simple. With the attribute `set_zero_at_surface` the origin is the surface face instead, as in pyOM2.

With `coord_degree` the coordinates x and y are longitude λ and latitude ϕ in degrees. The widths are given in degrees as well. The coordinates are built from the widths in degrees. After that the widths are converted to metres with the factor $a\pi/180$. The radius of the earth is $a = 6370\,\mathrm{km}$ (attribute `radius`). π is the double precision value. pyOM2 uses a single precision value, which gives relative differences of $3 \cdot 10^{-8}$ in all metric quantities.

The metric factors are $\cos\phi$ at the T rows (`cost`), $\cos\phi$ at the U rows (`cosu`) and $\tan\phi/a$ at the T rows (`tantr`). In Cartesian coordinates they are 1, 1 and 0. The horizontal areas of the cells are

$$A^t = \cos\phi^t\,\Delta x^t \Delta y^t \;, \quad A^u = \cos\phi^t\,\Delta x^u \Delta y^t \;, \quad A^v = \cos\phi^u\,\Delta x^t \Delta y^u \tag{2}$$

for the T, U and V cells, with Δx and Δy in metres.

The attributes `dx`, `dy` and `dz` are set to None after the grid is built. Code from pe_simple that still uses them therefore stops with an error.

The grid arrays are one-dimensional. With MPI every process holds its part including the halo points. With JAX every shard holds its own halo points, as all fields do in pe_simple. With JAX n_x and n_y must be divisible by the number of shards in x and y.

Discrete operators

All operators of pe_simple are written in flux form. In pe_realistic every flux is multiplied by the width of the face it crosses. Every flux divergence is divided by the volume of the cell. On a uniform Cartesian grid every operator reduces to the one of pe_simple. This is tested to round-off.

The vertical velocity at the top face of cell k follows from continuity,

$$w_k = -\sum_{k' \le k} \Delta z^t_{k'} \left(\frac{u_i - u_{i-1}}{\cos\phi^t_j\,\Delta x^t_i} + \frac{\cos\phi^u_j\,v_j - \cos\phi^u_{j-1}\,v_{j-1}}{\cos\phi^t_j\,\Delta y^t_j} \right) \tag{3}$$

with u and v of cell k' and the sum over the wet cells of the column.

The tracer fluxes are those of pe_simple. The meridional flux is multiplied by $\cos\phi^u$. The flux divergence is divided by $\cos\phi^t\Delta x^t$, $\cos\phi^t\Delta y^t$ and Δz^t. The Superbee scheme uses the Courant numbers $|u|\Delta t/(\cos\phi^t\Delta x^t)$, $|\cos\phi^u v|\Delta t/(\cos\phi^t\Delta y^t)$ and $|w|\Delta t/\Delta z^t$. The WENO scheme uses the coefficients of pe_simple, those of a uniform grid, as Oceananigans does. A uniform tracer then stays uniform and the content is conserved. Where the widths of the cells change, the face value is of lower than fifth order.

The momentum advection is that of pe_simple with volume transports. The transports through the faces are $u\Delta z^t\Delta y^t$, $v\Delta z^t\cos\phi^u\Delta x^t$ and wA^t. The divergence is divided by the volume $A^u\Delta z^t$ of the U cell or $A^v\Delta z^t$ of the V cell.

The Coriolis force averages fv to the U point and fu to the V point with the widths of the cells as weights, as in pyOM2,

$$(\partial_t u)_{cor} = \frac{1}{4\Delta x^u_i}\left(f_i\,\Delta x^t_i\,(v_{i,j}+v_{i,j-1}) + f_{i+1}\,\Delta x^t_{i+1}\,(v_{i+1,j}+v_{i+1,j-1})\right) \tag{4}$$

$$(\partial_t v)_{cor} = -\frac{1}{4\cos\phi^u_j\,\Delta y^u_j}\left(f_j\cos\phi^t_j\Delta y^t_j\,(u_{i,j}+u_{i-1,j}) + f_{j+1}\cos\phi^t_{j+1}\Delta y^t_{j+1}\,(u_{i,j+1}+u_{i-1,j+1})\right) \tag{5}$$

with the Coriolis parameter f at the T points. On a uniform grid this is the average of pe_simple.

On the sphere the momentum equations contain the metric terms $+uv\tan\phi/a$ and $-u^2\tan\phi/a$. They are discretised with the same weights as the Coriolis force,

$$(\partial_t u)_{met} = \frac{1}{8\Delta x^u_i}\left(\left(\frac{\tan\phi}{a}\,\Delta x^t (u_i+u_{i-1})(v_j+v_{j-1})\right)_i + (\cdots)_{i+1}\right) \tag{6}$$

$$(\partial_t v)_{met} = -\frac{1}{8\cos\phi^u_j\,\Delta y^u_j}\left(\left(\frac{\tan\phi}{a}\cos\phi^t\Delta y^t (u_i+u_{i-1})^2\right)_j + (\cdots)_{j+1}\right) \tag{7}$$

where $(\cdots)$ repeats the bracket before it at the next T point. In Cartesian coordinates both terms vanish.

The pressure gradients are $-(p_{i+1}-p_i)/(\cos\phi^t\Delta x^u_i)$ and $-(p_{j+1}-p_j)/\Delta y^u_j$. This holds for the hydrostatic pressure p_h and for the surface pressure p_s.

The hydrostatic pressure is integrated downward from the surface cell with the distances Δz^w,

$$p_h(n_z) = \frac{g}{2\rho_0}\rho(n_z)\,\Delta z^w(n_z)\ , \quad p_h(k) = p_h(k+1) + \frac{g}{2\rho_0}\left(\rho(k+1)+\rho(k)\right)\Delta z^w(k) \tag{8}$$

where n_z denotes the surface cell here.

The surface pressure of the rigid lid and of the implicit free surface follows from the vertically integrated continuity equation, as in pe_simple. The depth of the water column at the U and V points is $h^u = \sum\Delta z^t$ and $h^v = \sum\Delta z^t$ over the wet U and V cells. The

discrete equation for p_s is

$$\begin{aligned} & \frac{1}{\cos\phi^t \Delta x_i^t}\left(\frac{h_i^u(p_{i+1}-p_i)}{\cos\phi^t\Delta x_i^u} - \frac{h_{i-1}^u(p_i-p_{i-1})}{\cos\phi^t\Delta x_{i-1}^u}\right) \\ + & \frac{1}{\cos\phi_j^t \Delta y_j^t}\left(\frac{\cos\phi_j^u h_j^v(p_{j+1}-p_j)}{\Delta y_j^u} - \frac{\cos\phi_{j-1}^u h_{j-1}^v(p_j-p_{j-1})}{\Delta y_{j-1}^u}\right) = \frac{1}{\Delta t}\,\boldsymbol{\nabla}\cdot\int \boldsymbol{u}^* dz \end{aligned}$$

with the divergence on the right discretised as for w above. The implicit free surface adds $-p_s/(g\Delta t^2)$ on the left as in pe_simple. The equation is solved with the conjugate-gradient solver of pe_simple.

The rigid lid does not converge on a domain that is periodic in x and contains land. pyOM2 uses a streamfunction with island integrals there, which pe_realistic does not have. Such domains need the implicit free surface.

The vertical mixing of tracers and momentum uses the widths Δz^t and the distances Δz^w,

$$(\partial_t T)_{mix} = \frac{1}{\Delta z_k^t}\left(\kappa_k \frac{T_{k+1}-T_k}{\Delta z_k^w} - \kappa_{k-1}\frac{T_k - T_{k-1}}{\Delta z_{k-1}^w}\right)$$

with the coefficient κ_k at the top face of cell k. The stability limit of the explicit scheme becomes $4\max(\kappa, A_v)\Delta t/\min(\Delta z^t \Delta z^w) \le 1/2$.

Horizontal friction

Horizontal momentum friction uses the operators of pyOM2 (`harmonic_friction.f90` and `biharmonic_friction.f90`). They place the fluxes and the metric factors on the rows of the U and V points. pe_simple uses an operator at the T points for u and v instead. On a grid with varying Δy or on the sphere that operator is not exact for u and v.

For u the harmonic operator with coefficient A is

$$\begin{aligned} F_u &= \frac{f_i^x - f_{i-1}^x}{\cos\phi_j^t\,\Delta x_i^u} + \frac{f_j^y - f_{j-1}^y}{\cos\phi_j^t\,\Delta y_j^t} \\ f_i^x &= A\,(\cos\phi_j^t)^p\,\frac{u_{i+1}-u_i}{\cos\phi_j^t\,\Delta x_{i+1}^t}\;, \quad f_j^y = A\,(\cos\phi_j^u)^p\cos\phi_j^u\,\frac{u_{j+1}-u_j}{\Delta y_j^u} \end{aligned}$$

A flux exists only between two wet U cells.

For v the harmonic operator is

$$\begin{aligned} F_v &= \frac{f_i^x - f_{i-1}^x}{\cos\phi_j^u\,\Delta x_i^t} + \frac{f_j^y - f_{j-1}^y}{\cos\phi_j^u\,\Delta y_j^u} \\ f_i^x &= A\,(\cos\phi_j^u)^p\,\frac{v_{i+1}-v_i}{\cos\phi_j^u\,\Delta x_i^u}\;, \quad f_j^y = A\,(\cos\phi_{j+1}^t)^p\cos\phi_{j+1}^t\,\frac{v_{j+1}-v_j}{\Delta y_{j+1}^t} \end{aligned}$$

A flux exists only between two wet V cells.

The harmonic friction is $\boldsymbol{u} \leftarrow \boldsymbol{u} + \Delta t\,(F_u, F_v)$ with $A =$ `Ah`. The exponent p is `hor_friction_cosPower` (default 0). It is applied only in the rigid-lid and implicit free-surface scheme.

The biharmonic friction applies the harmonic operator twice, $\boldsymbol{u} \leftarrow \boldsymbol{u} - \Delta t\, F(F(\boldsymbol{u}))$. Each pass uses $A = \sqrt{A_{hbi}}$ with $A_{hbi} =$ `Ahbi`. The exponent p is `biha_friction_cosPower` (default 0). Since $(\cos\phi)^p$ enters each pass, the viscosity scales with $(\cos\phi)^{2p}$. In the split-explicit scheme the biharmonic friction enters the forcing of the external mode as the tendency $-F(F(\boldsymbol{u}))$ of the same operator.

The biharmonic mixing of tracers (`Khbi`) uses the operator at the T points with the metric factors of the tracer fluxes above.

The bottom friction is that of pe_simple. The thickness of the bottom cell is Δz^t of that cell instead of Δz.

Split-explicit external mode

The split-explicit scheme is that of pe_simple. The Coriolis force on the transports uses the weights of the Coriolis force above. The pressure gradient and the divergence of the transports carry the metric factors above.

On the sphere the external mode also contains the metric terms. They are the vertical integrals of $+uv\tan\phi/a$ and $-u^2\tan\phi/a$ with $u = U/h^u$ and $v = V/h^v$. They are discretised as the metric terms above and multiplied by h^u and h^v. The metric terms of the baroclinic part are already part of the Coriolis tendency of the three-dimensional velocity.

Equation of state

The attribute `eq_of_state` selects the equation of state. The value 0 means that temperature is buoyancy. The value 1 is the linear equation of state. The value 5 is the 48-term equation of state of TEOS-10 (GSW), as in pyOM2. The value 100 is the ideal gas of the atmosphere. The density is the anomaly to $\rho_0 = 1024\,\mathrm{kg/m^3}$, as in pyOM2.

The pressure in the equation of state of TEOS-10 is the depth of the cell below the surface face in dbar, as in pyOM2. It is z^w of the surface face minus z^t of the cell. It is not $z^t + \Delta z^t/2$, since z^t is not the midpoint of the cell on a stretched grid. The value 100 needs the default origin of z at the bottom.

N^2 for the value 5 uses the adiabatic displacement as in pe_simple. The density is evaluated once per time step as in pe_simple.

TKE closure

The TKE closure is that of pe_simple with the following changes. The mixing length sweeps use the widths Δz^t instead of Δz. The surface value of `mxl_choice` $= 1$ is limited by $\ell_{min} + z_0 + \Delta z^t$ of the surface cell. The vertical shear and N^2 use the distances Δz^w. The surface W cell has the thickness $\Delta z^w/2$ of the surface face.

The shear production K_{diss} is computed at the U and V points and averaged to the T points. The average is weighted with the areas, as in pyOM2,

$$K_i^t = \frac{A_i^u K_i^u + A_{i-1}^u K_{i-1}^u}{2A_i^t} \tag{9}$$

and correspondingly in y with A^v. An average without the areas would change K_{diss} by 6% at 60° latitude.

Isoneutral mixing

Isoneutral mixing is switched on by `enable_neutral_diffusion`. It is a port of `isoneutral.f90` and `isoneutral_diffusion.f90` of pyOM2. It uses the triads of Griffies et al., as pyOM2 does.

The isoneutral slopes follow from the gradients of temperature and salinity and the derivatives of density. For `eq_of_state` = 0 the derivative with respect to temperature is $-\rho_0/g$. pyOM2 uses zero there. For the value 1 the derivatives are the constants of the linear equation of state. For the value 5 they are the derivatives of the TEOS-10 equation of state. The value 100 is not possible.

Steep slopes are tapered with

$$\tau(s) = \frac{1}{2}\left(1 + \tanh\frac{s_c - |s|}{\Delta s}\right) \tag{10}$$

with s_c = `iso_slopec` (default 10^{-3}) and Δs = `iso_dslope` (default $8 \cdot 10^{-4}$).

The diffusivity is the three-dimensional field `K_iso`. It is set to the constant `K_iso_0` at the start unless the configuration has set the field. The horizontal diffusivity in the tensor is $\max(K_{steep}, K_{iso}\,\tau)$ with K_{steep} = `K_iso_steep`. The field can be changed during the run, for instance by another parameterisation.

With `enable_skew_diffusion` the skew flux of Gent and McWilliams is added, as in pyOM2. Its diffusivity is the three-dimensional field `K_gm`. It is set to the constant `K_gm_0` at the start unless the configuration has set the field. On the lateral faces the triads then use $K_{iso} - K_{gm}$. On the top faces they use $K_{iso} + K_{gm}$.

The horizontal and mixed parts of the tensor are applied with a forward Euler step. The vertical part K_{33} is applied with the implicit vertical diffusion of the model. Isoneutral mixing acts on temperature and, for an equation of state with salinity, on salinity.

At the start the model checks the stability of the explicit part, as pyOM2 does. The quantity

$$\delta = \min\left(\frac{\cos\phi^t \Delta x^t \Delta z^t}{4K_{iso,0}\Delta t},\ \frac{\Delta y^t \Delta z^t}{4K_{iso,0}\Delta t}\right)$$

is the steepest slope that is stable. The model stops with an error if δ is smaller than s_c.

The following parts of pyOM2 are not ported. These are the dissipation diagnostics of the isoneutral and skew fluxes, the treatment in the mixed layer and the friction of the TEM formulation.

Tracer advection outside the Adams–Bashforth step

With thin cells on a fine horizontal grid the vertical Courant number $|w|\Delta t/\Delta z^t$ can limit the time step. The Superbee scheme is stable up to a Courant number of 1 in a forward step. Inside the Adams–Bashforth step with the weights 1.6 and -0.6 it becomes unstable above about 0.45. The attribute `vadv_sub_cycling` $= M$ therefore takes the tracer advection out of the Adams–Bashforth step. The default $M = 0$ keeps the advection in the Adams–Bashforth step.

With $M > 0$ the advection uses the velocities u, v and w at the end of the time step. It consists of a step in x, a step in y and M steps in z with the time step $\Delta t/M$. Each of them is a forward step of the Superbee scheme, as in MITgcm. The vertical Courant number may then be as large as M. The Adams–Bashforth step adds only the forcing of the tracers. The option needs `superbee_advection`. It cannot be combined with `weno_advection`, and the model stops with an error then. The steps outside the Adams–Bashforth step are forward steps of the Superbee scheme. The WENO scheme is written for the Adams–Bashforth step.

All of the advection has to leave the Adams–Bashforth step, not the vertical part alone. Horizontal and vertical advection cancel in a flow along isopycnals. A step in which one of them is forward and the other is part of the Adams–Bashforth step is unstable in such a flow. For the same reason the vertical advection of momentum stays in the Adams–Bashforth step.

The velocities at the end of the time step are needed. With the velocities at the start of the time step the forward tracer step and the Adams–Bashforth step of momentum amplify internal waves.

A step in one direction changes a uniform tracer, since the flow in that direction alone is not free of divergence. Each step therefore carries the volume ϱ that the cell would have after the steps done so far (Easter 1993, Lin and Rood 1996). It starts with $\varrho = 1$ and changes in each step in x as

$$\varrho \leftarrow \varrho - \frac{\Delta t}{\cos\phi^t \Delta x^t} \left(u_i - u_{i-1}\right)$$

and correspondingly in y and z. The fluxes are formed with the tracer value $T = q/\varrho$ and change the tracer content q. The content is conserved to round-off, and a uniform tracer stays uniform.

The flux through the surface face is handled in the steps in z. It is zero, or $w\,T$ of the surface cell with `free_surface_tracer_flux`. Its mean over the M steps is kept in `vadv_fs_flux` for a configuration that books the tracer budget.

The option adds no new state to the restart. The tendencies `dtemp` and `dsalt` then contain the forcing only. The first step after switching the option on or off should therefore not use the old tendencies.

Time step

The order of the calls in a time step of the split-explicit scheme is

1. diffusivities of the TKE closure (`set_tke_diffusivities`)
2. momentum tendencies, Coriolis force and hydrostatic pressure (`momentum_tendency_expl`)
3. forcing of the configuration (`set_forcing`)
4. bottom friction
5. external mode with sub cycling (`solve_expl_surface`)
6. vertical friction
7. vertical velocity
8. tracer advection and biharmonic tracer mixing, or the advection outside the Adams–Bashforth step
9. Adams–Bashforth update of the tracers
10. isoneutral mixing
11. vertical mixing of the tracers
12. density
13. TKE step (`integrate_tke`)

The order of the calls in a time step of the rigid lid and of the implicit free surface is

1. diffusivities of the TKE closure
2. tracer advection and biharmonic tracer mixing
3. momentum tendencies and hydrostatic pressure (`momentum_tendency_impl`)
4. forcing of the configuration (`set_forcing`)
5. biharmonic and harmonic momentum friction
6. Adams–Bashforth update of the velocities
7. direct forcing of the configuration (`set_forcing_direct`)
8. surface pressure and its gradient (`solve_for_pressure`)
9. vertical friction
10. bottom friction
11. vertical velocity

12. Adams–Bashforth update of the tracers, or the advection outside the Adams–Bashforth step
13. isoneutral mixing
14. vertical mixing of the tracers
15. density
16. TKE step

In both schemes the steps are those of pe_simple, with the isoneutral mixing, the harmonic friction and the advection outside the Adams–Bashforth step added.

Model parameters and options

The parameters and options of pe_simple are also those of pe_realistic. The tables list only the differences.

Required parameters

Attribute	Unit	Meaning
`dxt`, `dyt`	m or °	widths of the cells in x and y, a number or an array without halo. Degrees with `coord_degree`. Replace `dx` and `dy` of pe_simple, which are still accepted
`dzt`	m	widths of the cells in z, a number or an array without halo, from the bottom to the surface. Replaces `dz` of pe_simple, which is still accepted
`eq_of_state`	—	equation of state, 0, 1, 5 (TEOS-10) or 100

The configuration must set the Coriolis parameter with `set_coriolis`. The field `coriolis_t` is zero otherwise.

Options with defaults

Attribute	Default	Meaning
`coord_degree`	off	longitude and latitude in degrees instead of Cartesian coordinates
`x_origin`, `y_origin`	0	position of the first interior U point
`radius`	6370 km	radius of the earth (m)

Attribute	Default	Meaning
`set_zero_at_surface`	off	origin of z at the surface face instead of the bottom face
`Ah`	unset	harmonic momentum friction (m^2/s), active if present. Rigid lid and implicit free surface only
`hor_friction_cosPower`	0	exponent p of $(\cos\phi)^p$ in the harmonic friction
`biha_friction_cosPower`	0	exponent p of $(\cos\phi)^p$ in each pass of the biharmonic friction
`enable_neutral_diffusion`	off	isoneutral mixing
`enable_skew_diffusion`	off	skew flux of Gent and McWilliams
`K_iso_0`	0	isoneutral diffusivity (m^2/s), initial value of `K_iso`
`K_gm_0`	0	skew diffusivity (m^2/s), initial value of `K_gm`
`K_iso_steep`	0	lower limit of the tapered horizontal diffusivity (m^2/s)
`iso_slopec`	10^{-3}	slope at which the taper is one half
`iso_dslope`	$8 \cdot 10^{-4}$	width of the taper
`K_iso`, `K_gm`	unset	three-dimensional diffusivities, set from `K_iso_0` and `K_gm_0` if not given
`vadv_sub_cycling`	0	number M of vertical steps of the tracer advection outside the Adams–Bashforth step. 0 keeps the advection in the Adams–Bashforth step
`weno_advection`	off	as in pe_simple, with the coefficients of a uniform grid. Not together with `vadv_sub_cycling`
`weno_ref`	1	as in pe_simple

Snapshot diagnostics

The snapshot classes `snap_nc4` and `snap_raw` are those of pe_simple. They write the coordinates of the grid above. These are x^t, x^u, y^t, y^u, z^t and z^w, in degrees with `coord_degree`. z^w is written under the name `zu`.

Limits

- The grid must not include the poles. The zonal width $\cos\phi\,\Delta x$ goes to zero there, and $\tan\phi$ grows without bound.
- The rigid lid does not converge on a domain that is periodic in x and contains land.

- TEM friction, the EKE model and IDEMIX of pyOM2 are not part of pe_realistic.
- With varying Δx^t in a periodic direction pe_realistic and pyOM2 build different x^t and Δx^u in the halo.
- Δz^t must vary smoothly, see the grid above.
- `weno_advection` and `vadv_sub_cycling` exclude each other.